



**UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH**

**Escola Superior d'Enginyeries Industrial,
Aeroespacial i Audiovisual de Terrassa**

Bachelor's Degree in Aerospace Vehicle Engineering

Transient-state case of conduction heat transfer in a composite wall with constant cross-section fins

Gas dynamics and heat and mass transfer

Marina Liao Colom Escolà

January 11, 2026

Contents

1	Studied case description	2
2	Analytical resolution	3
3	Numerical resolution: Gauss-Seidel method	4
4	Steady-state verification	6
5	Transient-state verification	8
6	Code implementations	9
6.1	Natural convection	9
6.2	Forced convection	9
6.3	Infrared radiation	9
7	Conclusions	11
8	Conclusions	11
9	Bibliography	12
	Appendices	13
A	Main code	13
B	Previous calculations code	15
C	Coefficient calculation code	18
D	Solver code	23

1 Studied case description

Through this practice, a case of conduction heat transfer in transient-state has been studied. Specifically, a pair of walls of height H and thickness W , each one composed of two materials, joined by N_f constant cross-section fins of length L_f and thickness e_f (Figure 1) have been analyzed.

Being $k \in \{A, B, C, D, f\}$, each segment e_k has thermal conductivity λ_k . Fins also dissipate heat through forced convection with a constant heat transfer coefficient of α_{ext} . This convection heat is assumed one-dimensional along the y-axis. In addition, there is an external heat source at the left end of the wall $\dot{q}_A(t)$ and at the right end $\dot{q}_D(t)$.

The system is in transient-state, temperature is assumed one-dimensional along the x-axis – $T(x)$ – and heat is assumed time-dependent and two-dimensional – $Q(t, x, y)$ –. There is heat generation in each segment – $\dot{q}_{vA}(t) = \dot{q}_{vB}(t) = \dot{q}_{vC}(t) = \dot{q}_{vD}(t)$ – and no convection heat transfer besides in fins surfaces S_{conv} . It also can be assumed a perfect thermal contact between the two materials and fins and a constant thermal conductivity in each material. Moreover, all non-mentioned faces have been considered adiabatic and thermophysical properties – $\rho_k, c_{pk}, \lambda_k$ – are known.

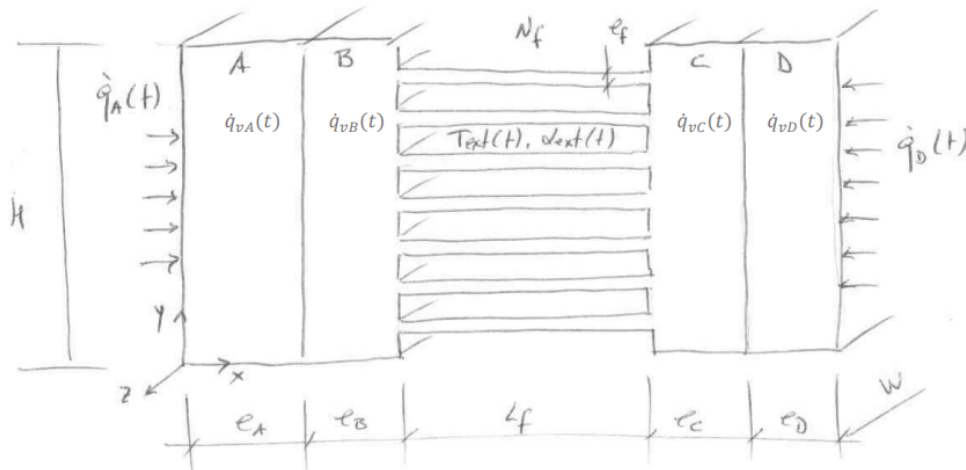


Figure 1: Case sketch [CTT]

2 Analytical resolution

For the analytical solution of the problem, an energy balance is performed on each interface of the composite wall. In addition, the conduction heat for fins is defined in order to determine the following system of equations needed to solve the problem.

$$\begin{cases} F_0 = q_A + \lambda_A \frac{T_1 - T_0 + \frac{q_{v0}}{2\lambda_A} e_A^2}{e_A} = 0 \\ F_1 = q_{v0} e_A - \lambda_A \frac{T_1 - T_0 + \frac{q_{v0}}{2\lambda_A} e_A^2}{e_A} - q_{v1}(e_A - e_A) + \lambda_B \frac{T_2 - T_1 + \frac{q_{v1}}{2\lambda_B} e_B^2}{e_B} = 0 \\ F_2 = (q_{v1} e_B - \lambda_B \frac{T_2 - T_1 + \frac{q_{v1}}{2\lambda_B} e_B^2}{e_B}) S - \alpha_g (T_2 - T_g) S_{f2} - Q_{f2} = 0 \\ F_3 = \alpha_g (T_g - T_3) S_{f32} + Q_{f3} + (-q_{v3} 0 - \lambda_C \frac{T_4 - T_3 + \frac{q_{v3}}{2\lambda_C} e_C^2}{e_C}) S = 0 \\ F_4 = q_{v3} e_C - \lambda_C \frac{T_4 - T_3 + \frac{q_{v3}}{2\lambda_C} e_C^2}{e_C} - q_{v4} 0 + \lambda_D \frac{T_5 - T_4 + \frac{q_{v4}}{2\lambda_D} e_D^2}{e_D} = 0 \\ F_5 = q_{v4} e_D - \lambda_D \frac{T_5 - T_4 + \frac{q_{v4}}{2\lambda_D} e_D^2}{e_D} + q_B = 0 \\ F_6 = Q_{f2} + \lambda_f m (C_1 - C_2) e_f W n_{fin} = 0 \\ F_7 = Q_{f3} + \lambda_f m (C_1 e^{mL_f} - C_2 e^{-mL_f}) e_f W n_{fin} = 0 \\ F_8 = T_2 - (T_g + C_1 + C_2) = 0 \\ F_9 = T_3 - (T_g + C_1 e^{mL_f} + C_2 e^{-mL_f}) = 0 \end{cases}$$

Temperature T_x corresponds to the interface between materials $x \in N \cap [0, 5]$ numbered from left to right according to Figure 1. By solving this system, the temperature field below and constants C_1 and C_2 can be obtained.

$$T(x) = \begin{cases} T_0 + \frac{T_1 - T_0 + \frac{q_{v0}}{2\lambda_A} e_A^2}{e_A} x - \frac{q_{v0}}{2\lambda_A} x^2 & x \in [0, e_A] \\ T_1 + \frac{T_2 - T_1 + \frac{q_{v1}}{2\lambda_B} e_B^2}{e_B} (x - e_A) - \frac{q_{v1}}{2\lambda_B} (x - e_A)^2 & x \in [e_A, e_A + e_B] \\ T_{ext} + C_1 e^{mx} + C_2 e^{-mx} & x \in [e_A + e_B, e_A + e_B + L_f] \\ T_3 + \frac{T_4 - T_3 + \frac{q_{v3}}{2\lambda_C} e_C^2}{e_C} (x - (e_A + e_B + L_f)) - \frac{q_{v3}}{2\lambda_C} (x - (e_A + e_B + L_f))^2 & x \in [e_A + e_B + L_f, e_A + e_B + L_f + e_C] \\ T_4 + \frac{T_5 - T_4 + \frac{q_{v4}}{2\lambda_D} e_D^2}{e_D} (x - (e_A + e_B + L_f + e_C)) - \frac{q_{v4}}{2\lambda_D} (x - (e_A + e_B + L_f + e_C))^2 & x \in [e_A + e_B + L_f + e_C, e_A + e_B + L_f + e_C + e_D] \end{cases}$$

This system can be solved by an iterative method such as Gauss-Seidel. In this report, the developed code (see Appendices A, B, C and D) uses Gauss-Seidel to solve the discretized equations as explained in the following section.

3 Numerical resolution: Gauss-Seidel method

For the numerical resolution of this problem, the Crank-Nicholson scheme has been used – $\beta = 0.5$ – to ensure a positive balance between numerical stability and precision.

The domain can be discretized in $N = N_A + N_B + N_F + N_C + N_D$ control volumes. By making an uniform mesh for each segment, N nodes can be located in the center of each control volume at given distances:

$$\Delta x_A = \frac{e_A}{N_A} \quad \Delta x_B = \frac{e_B}{N_B} \quad \Delta x_F = \frac{L_f}{N_F} \quad \Delta x_C = \frac{e_C}{N_C} \quad \Delta x_D = \frac{e_D}{N_D}$$

Also, two external nodes are defined at the beginning ($x = 0$) and end ($x = e_A + e_B + L_f + e_C + e_D$) of the system, having a total of $N + 2$ nodes. Thus, the position along the x-axis of the control volumes $x_{VC}[i]$ and the nodes $x[i]$ can be defined as:

$$\begin{aligned} x_{VC}[i] &= (i-1)\Delta x & \text{for } i &= [1, N] \\ x[1] &= 0 \\ x[i] &= \frac{x_{VC}[i] + x_{VC}[i-1]}{2} & \text{for } i &= [2, N+1] \\ x[N+2] &= 2e_n + 2e_n + L_a \end{aligned}$$

Thereafter, it is determined the equation for the problem resolution as:

$$\beta Q_p^{n+1} + (1-\beta)Q_p^n = \rho c_p \frac{T_p^{n+1} - T_p^n}{\Delta t} V_p$$

where n is an instant of time, Δt is the time step and V is the volume.

Accordingly, the discretized equation for each node in a given instant of time is:

$$a_P[i]T_P[i] = a_E[i]T_E[i-1] + a_W[i]T_W[i+1] + b_P[i]$$

being

$$\begin{aligned} \text{for } i = 1 & \begin{cases} a_W[1] = 0 \\ a_E[1] = \beta \frac{\lambda_A}{d_{PE}} S_e \\ a_P[1] = a_E[1] \\ b_P[1] = \beta q_A S_e + (1-\beta)[q_A^{old} + \lambda_A \frac{T^n[2] - T^n[1]}{d_{PE}}] S_e \end{cases} \\ \text{for } i = [2, N+1] & \begin{cases} a_W[i] = \beta \frac{\lambda_W}{d_{PW}} S_w \\ a_E[i] = \beta \frac{\lambda_E}{d_{PE}} S_e \\ a_P[i] = a_E[i] + a_W[i] + \frac{\rho_k V_p c_p}{\Delta t} + \beta \alpha_{ext} S_{conv} \\ b_P[i] = \beta q_{vk} V_p + \frac{\rho_k V_p c_p}{\Delta t} T^n[i] + (1+\beta)[q_{vk}^{old} V_p + \\ \quad \frac{\lambda_E S_e}{d_{PE}} (T^n[i+1] - T^n[i]) - \frac{\lambda_W S_w}{d_{PW}} (T^n[i] - T^n[i-1]) - \\ \quad \alpha_{ext} S_{conv} (T^n[i] - T_{ext})) + \beta \alpha_{ext} S_{conv} T_{ext} \end{cases} \\ \text{for } i = N+2 & \begin{cases} a_W[N+2] = \beta \frac{\lambda_D}{d_{PW}} S_w \\ a_E[N+2] = 0 \\ a_P[N+2] = a_W[N+2] \\ b_P[N+2] = \beta q_D S_w + (1-\beta)[q_D^{old} - \lambda_D \frac{T^n[N+2] - T^n[N+1]}{d_{PW}}] S_w \end{cases} \end{aligned}$$

where

$$\text{for nodes in material } k = A, B, C \text{ or } D \begin{cases} \lambda_W = \lambda_E = \lambda_k \\ S_w = S_e = H \cdot W \\ V_p = (x[i] - x[i-1])H \cdot W \\ S_{conv} = 0 \end{cases}$$

$$\text{for nodes in fins } k = f \left\{ \begin{array}{l} \lambda_W = \lambda_E = \lambda_f \\ S_w = S_e = N_f e_f W \\ V_p = N_f (x[i] - x[i-1]) e_f W \\ S_{conv} = 2N_f (x[i] - x[i-1]) W \\ q_{vf} = q_{vf}^{old} = 0 \end{array} \right.$$

As the wall is made of different materials, distances between the center of the control volume and east and west faces are not equal. Therefore, the following distances are considered for nodes located between materials:

$$d_{Pe} = x_{VC}[i] - x[i]$$

$$d_{eE} = x[i+1] - x_{VC}[i]$$

$$d_{Pw} = x[i] - x_{VC}[i-1]$$

$$d_{wW} = x_{VC}[i-1] - x[i-1]$$

Furthermore, the specific heat c_p is obtained as a result of the current $-n-$ and previous $-n+1-$ values:

$$c_p = c_{pk} \left(\frac{T^{n+1}[i] + T^n[i]}{2} \right)$$

Also, λ 's are the harmonic thermal conductivities between two materials $-j-$ and $l-$ that can be written as:

$$\lambda_{harm(j,l)} = \frac{d_{PE}}{\frac{d_{Pe}}{\lambda_j} + \frac{d_{eE}}{\lambda_l}}$$

same with the west face.

The algorithm for the numerical resolution is described as:

1. Data entry: physical ($L_f, e_k, \lambda_k, \rho_k, c_{pk}, W...$) and numerical ($N_k, \Delta t, \delta, T_{start}, \beta...$) known data.
2. Definition of vectors and previous calculations: e.g. $x[i], x_{VC}[i], S_{conv}$.
3. Initial map of temperatures $T^n[i] = T_{start}$ with n being $t = t_0$.
 - (a) Initial map of the time loop: $T^{n+1}[i] = T^n[i]$ with $n+1$ being $t + \Delta t$
 - (b) Calculation of the coefficients for external nodes: $a_P[1], a_W[1], a_E[1], b_P[1]$ and $a_P[N+2], a_W[N+2], a_E[N+2], b_P[N+2]$.
 - (c) Calculation of the coefficients for internal nodes: $a_P[i], a_W[i], a_E[i], b_P[i]$ for $i = [2, N+1]$.
 - (d) Gauss-Seidel solver:

$$T[i] = \frac{a_E[i]T^*[i+1] + a_W[i]T[i-1] + b_P[i]}{a_P[i]}$$
 - (e) Is $\max|T[i] - T^*[i]| < \delta$?
 NO: $T^{n+1}[i] = T[i]$ and point 3.b.
 YES: point 4.
4. Is $n = n_{max}$ or $\max|T^n[i] - T^{n+1}[i]| < \delta$?
 NO: $n = n+1$ ($t = t + \Delta t$) and point 3.a.
 YES: point 5.
5. Final calculations and print results.
6. End

4 Steady-state verification

In order to verify the correct problem resolution through the developed code, data from exercise 4 (Tables 1 and 4) of the dossier of fins with analytical treatment [ETS] have been used as input.

Table 1: General data, material properties, and geometry

Parameter	Value
Number of fins (N_{fin})	100
Initial temperature (T_0)	273 K
Time step (Δt)	1 s
Convergence tolerance	1×10^{-20}
δ	1×10^{-5}
External convection coefficient (α_{ext})	70 W/m ² K
Ambient temperature (T_{ext})	293 K
Specific heat capacity (C_p)	0
Crank–Nicolson parameter (β)	1
Thermal conductivity A (λ_A)	50 W/mK
Thermal conductivity B (λ_B)	220 W/mK
Thermal conductivity C (λ_C)	220 W/mK
Thermal conductivity D (λ_D)	50 W/mK
Thermal conductivity F (λ_F)	220 W/mK
Thickness e_A	0.01 m
Thickness e_B	0.005 m
Thickness e_C	0.005 m
Thickness e_D	0.01 m
Thickness e_F	0.002 m
Nodes in region A (N_A)	30
Nodes in region B (N_B)	10
Nodes in region C (N_C)	10
Nodes in region D (N_D)	30
Nodes in fins (N_F)	50
Fin length (L_f)	0.4 m
Height (H)	0.5 m
Width (W)	0.5 m

Table 2: Heat source data

Source	Value
Surface heat flux in region A (q_A)	5000 W/m ²
Surface heat flux in region D (q_D)	50000 W/m ²
Volumetric heat source in region A (q_{VA})	0 W/m ³
Volumetric heat source in region B (q_{VB})	0 W/m ³
Volumetric heat source in region C (q_{VC})	0 W/m ³
Volumetric heat source in region D (q_{VD})	0 W/m ³

The expected results from the analytical resolution from exercise 4 [ETS] are:

$$T_0 = 24.26K \quad T_1 = 23.26K$$

$$T_2 = 23.15K \quad T_3 = 51.03K$$

$$T_4 = 52.16K \quad T_5 = 62.16K$$

And the obtained results from the developed code (Figure 2) are:

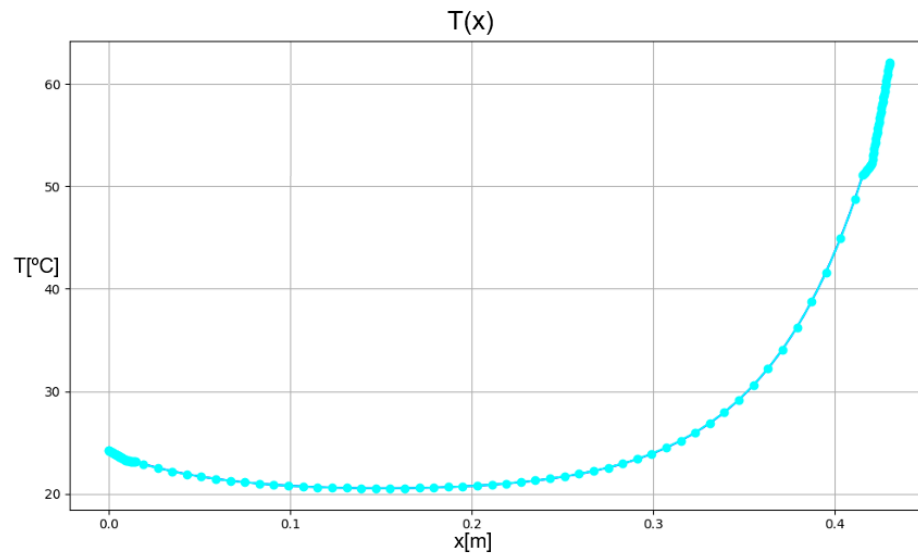


Figure 2: $T(x)$ obtained from code

The observed maximum relative error is on the order of $10^{-2}\%$. Because of this small value, the code is validated.

5 Transient-state verification

By adding the following values from Tables 3 and 4 to the data in Table 1, the transient state case is evaluated through the code but using the Crank-Nicholson method $\beta = 0.5$.

Table 3: Specific heat capacities of the materials

Material	Specific heat capacity, C_p (J/(kg·K))
Material A	500
Material B	700
Material C	250
Material D	200
Material F	660

Table 4: Heat source data

Source	Value
Surface heat flux in region A (q_A)	$1000 \sin(\pi \frac{t}{60})$ W/m ²
Surface heat flux in region D (q_D)	50000 W/m ²
Volumetric heat source in region A (q_{VA})	$5 \cdot 10^6$ W/m ³
Volumetric heat source in region B (q_{VB})	$5 \cdot 10^6$ W/m ³
Volumetric heat source in region C (q_{VC})	$5 \cdot 10^6$ W/m ³
Volumetric heat source in region D (q_{VD})	$5 \cdot 10^6$ W/m ³

Results obtained from the code are shown in Figure 3 below.

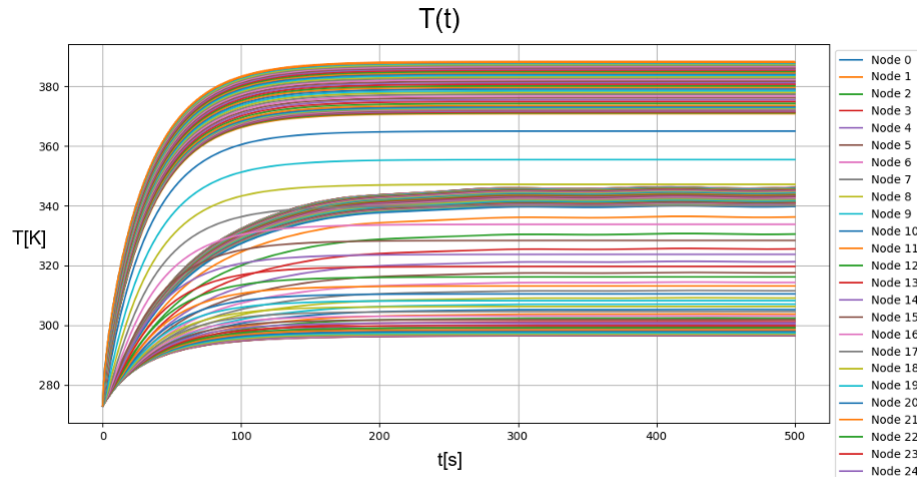


Figure 3: $T(t)$ obtained from code

It can be seen the time evolution of the temperature $T(t)$ for multiple nodes, where all nodes start from an initial temperature of $T_{start} = 273$ K and increase over time. Each curve corresponds to a different node and displays a rapid temperature rise at early times followed by a gradual approach to a steady-state value, indicating a transient heating process with asymptotic stabilization. The steady-state temperatures vary among nodes, forming distinct bands that indicates differences in heating conditions or boundary effects within the system.

The maximum relative error for the calculated temperature upon reaching steady-state is observed to be no greater than 1%. Therefore, the code is validated for the transient-state case too.

6 Code implementations

In order to study the proposed case in more depth, the following heat transfers have been added to the code.

6.1 Natural convection

Considering natural convection through the surfaces of the system, the coefficients are redefined as:

$$a_W[i] = \beta \frac{S_W}{d_{PW}} \lambda_W$$

$$a_E[i] = \beta \frac{S_E}{d_{PE}} \lambda_E$$

$$a_P[i] = a_E[i] + a_W[i] + \frac{\rho V_p c_{p,p}}{\Delta t} + \beta \alpha_{conv,nat} S_{conv,2} + \beta \alpha_{ext} S_{conv}$$

$$b_P[i] = \beta q_v V_p + \frac{\rho V_p c_{p,p}}{\Delta t} T^n[i] + (1-\beta)(q_v^{old} V_p + \lambda_E \frac{T^n[i+1] - T^n[i]}{d_{PE}} S_E - \lambda_W \frac{T^n[i] - T^n[i-1]}{d_{PW}} S_W - \alpha_{ext}(T^n[i] - T_{ext}) S_{conv} - \alpha_{conv,nat}(T^n[i] - T_{ext}) S_{conv,2}) + \alpha_{ext} T_{ext} S_{conv} \beta + \alpha_{conv,nat} \beta T_{ext} S_{conv,2}$$

where $\alpha_{conv,nat}$ is the natural convection coefficient according to the local temperature and $S_{conv,2}$ is the surface exposed to this convection.

The difference between both cases – without natural convection and with it – produce a maximum relative error of approximately 0.3%. In other words, this assumptions practically does not affect the results obtained.

6.2 Forced convection

For this case, the convective heat coefficient is calculated

$$\alpha_F = \frac{Nu \lambda}{x}$$

according to

$$Nu = \begin{cases} 0.332 Re^{1/2} Pr^{1/3} & \text{for } Re < 5 \cdot 10^5 \\ 0.029 Re^{4/5} Pr^{1/3} & \text{for } Re \geq 5 \cdot 10^5 \end{cases}$$

where

$$Re = \frac{\rho v_{ext} x}{\mu}$$

and

$$Pr = \begin{cases} \frac{\mu c_p}{\lambda} & \text{for } T_m < 1100 \\ 0.71 & \text{for } T_m \geq 1100 \end{cases}$$

being T_m the average temperature between the wall and the outside gas of the system.

Without this implementation and with it, the maximum relative error remains in the order of $10^{-1}\%$ as seen with the natural convection.

6.3 Infrared radiation

Taking into account infrared radiation, the coefficients are redefined as:

$$a_W[i] = \beta \frac{S_W}{d_{PW}} \lambda_W$$

$$a_E[i] = \beta \frac{S_E}{d_{PE}} \lambda_E$$

$$\begin{aligned}
a_P[i] &= a_E[i] + a_W[i] + \frac{\rho V_p c_{p,p}}{\Delta t} + \beta \alpha_{conv,nat} S_{conv,2} + \beta \alpha_{ext} S_{conv} + \beta \epsilon \sigma T^{n+1}[i]^3 S_{conv,2} \\
b_P[i] &= \beta q_v V_p + \frac{\rho V_p c_{p,p}}{\Delta t} T^n[i] + (1-\beta)(q_v^{old} V_p + \lambda_E \frac{T^n[i+1] - T^n[i]}{d_{PE}} S_E - \lambda_W \frac{T^n[i] - T^n[i-1]}{d_{PW}} S_W - \alpha_{ext}(T^n[i] - T_{ext}) S_{conv} \\
&\quad - \alpha_{conv,nat}(T^n[i] - T_{ext}) S_{conv,2} - \epsilon \sigma (T^n[i]^4 - T_{sky}^4) S_{conv,2}) + \alpha_{ext} T_{ext} S_{conv} \beta + \alpha_{conv,nat} \beta T_{ext} S_{conv,2} + \beta \epsilon \sigma T_{sky}^4 S_{conv,2}
\end{aligned}$$

The added radiation has more impact in the system than the natural or forced convection. Despite this, the maximum relative error obtained is no greater than 0.3% so the presented scheme is still valid due to the small relative error in comparison with the analytical resolution.

7 Conclusions

8 Conclusions

Through this study, a transient-state heat conduction problem in a composite wall with constant cross-section fins has been successfully analyzed by combining analytical modeling and numerical simulation. The physical problem was formulated under well-defined assumptions regarding material properties, heat generation, and boundary conditions, which allowed the derivation of a consistent analytical solution for the temperature distribution across the system. This formulation provided a solid reference framework against which the numerical results could be assessed.

A numerical solution based on the finite volume method and the Gauss–Seidel iterative scheme was implemented using the Crank–Nicolson time discretization. This approach proved to be both stable and accurate for the studied configurations, enabling the resolution of the coupled transient conduction problem across multiple materials and fins. The use of harmonic mean conductivities at material interfaces and temperature-dependent thermophysical properties ensured a physically coherent representation of heat transfer within the composite structure.

The validation of the numerical model was carried out through steady-state and transient-state verifications. In the steady-state case, the numerical results closely matched the analytical solution, with maximum relative errors on the order of $10^{-2}\%$, confirming the correctness of the discretization and solution strategy. For the transient-state analysis, the temporal evolution of temperature showed physically consistent behavior, characterized by an initial rapid heating phase followed by an asymptotic approach to steady-state conditions. The relative error upon reaching steady state remained below 1%, further validating the robustness of the implemented scheme.

Additional physical phenomena, including natural convection, forced convection, and infrared radiation, were incorporated into the numerical model to assess their influence on the thermal response of the system. The inclusion of natural and forced convection produced only minor variations in the temperature field, with relative differences remaining below 0.3%, indicating a limited impact under the studied conditions. In contrast, infrared radiation exhibited a more noticeable effect on the heat balance, although the overall accuracy of the numerical solution was preserved, with errors remaining within acceptable limits.

Overall, the developed numerical tool has demonstrated strong reliability and flexibility for the analysis of transient heat conduction problems in composite systems with fins. The agreement with analytical solutions and the consistent behavior under extended physical modeling confirm its suitability for more complex thermal analyses in engineering applications, particularly in the context of aerospace heat transfer problems.

9 Bibliography

References

- [CTT] CTTC (UPC). *Pràctiques de mètodes numèrics en Dinàmica de Gasos i Transferència de Calor i Massa*. Course notes. Teaching material, PDF. Centre Tecnològic de Transferència de Calor, Universitat Politècnica de Catalunya.
- [ETS] ETSEIAT–CTTC. *Proposal of Exercises in Conduction Heat Transfer Using Analytical Methods*. Teaching material. Unpublished course notes, PDF. ETSEIAT–CTTC, Universitat Politècnica de Catalunya (UPC).

Appendices

A Main code

main.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <math.h>
4
5  #include "headers/presolver.h"
6  #include "headers/coeff.h"
7  #include "headers/solver.h"
8
9  int main(){
10     //1.ENTRADA DE DADES
11     //fisiques
12     const int Na = 30, Nb = 10, Nc = 10, Nd = 30, Nf = 50;
13     const double ea = 0.01, eb = 0.005, ec = 0.005, ed = 0.01, ef = 0.0;
14     const double Lf = 0.4, H = 0.5, W = 0.5, h=0.05, rhoF = 9100.0;
15     const double vext = 20.0, alpha_ext = 70.0, Text = 293.0, Pext = 1e5;
16     const double Tsky = 268.0, epsilon = 0.9, sigma = 5.67e-8, lambda_F = 220.0;
17     const double rhoA = 6200.0, rhoB = 7800.0, rhoC = 8800, rhoD = 6700;
18     const double lambda_A = 50.0, lambda_B = 220.0, lambda_C = 220.0, lambda_D = 50.0;
19     //numeriques
20     const int nmax = 1200;
21     const double Tstart = 273.0;
22     const double delta = 5e-5, deltat = 0.5, beta = 0.5;
23     double maxdiff = 1e20, critFinal = 1e9, Mdiff=0, cFi=0;
24
25     //2.DEFINICIO DE VECTORS I CALCULS PREVIS
26     int crit = 0, n = 0, numfin = 100;
27     double t = 0, told,diff,cFinal;
28     int n_total = Na + Nb + Nc + Nd + Nf, Ntot = n_total + 2;
29     double *Qrad = (double*)malloc((Ntot)*sizeof(double));
30     double *QconvNat = (double*)malloc((Ntot)*sizeof(double));
31     double *aE = (double*)malloc((Ntot)*sizeof(double));
32     double *aW = (double*)malloc((Ntot)*sizeof(double));
33     double *aP = (double*)malloc((Ntot)*sizeof(double));
34     double *bP = (double*)malloc((Ntot)*sizeof(double));
35     double *Told = (double*)malloc((Ntot)*sizeof(double));
36     double *Tval = (double*)malloc((Ntot)*sizeof(double));
37     //malla
38     double delta_xA = ea/Na, delta_xB = eb/Nb, delta_xC = ec/Nc;
39     double delta_xD = ed/Nd, delta_xF = ef/Nf;
40     double *x = (double*)malloc((n_total+1)*sizeof(double));
41     double *x_nodes = (double*)malloc((Ntot)*sizeof(double));
42     double **T = (double**)malloc((nmax)*sizeof(double));
43     for(int i=0;i<nmax;i++){
44         T[i]=malloc(Ntot*sizeof(double));
45     double *Tcomp = (double*)malloc((Ntot)*sizeof(double));
46     mesh(x,n_total,Na,Nb,Nc,Nd,Nf,delta_xA,delta_xB,delta_xC,delta_xD,delta_xF,x_nodes);
47     //factors de radiacio
48     double F23, F24, F2sky, F32, F34, F3sky, F42, F43, F4sky;
49     FV(W,h,Lf,&F23,&F24,&F2sky,&F32,&F34,&F3sky,&F42,&F43,&F4sky);
50
51     //3.
52     //3.1.MAPA INICIAL DE TEMPERATURES
53     for(int i=0;i<Ntot;i++){
54         Tcomp[i]=Tstart;
55         T[0][i]=Tcomp[i];
56     }
57     //3.2.COEFFICIENTS
58     while(crit==0)
59     {
60         told=t;

```

```

61     t=t+deltat;
62     while(maxdiff>delta){
63         clear(aE); clear(aW); clear(aP); clear(bP);
64         //coefficients dels nodes externs
65         coeff_boundary(H, W, beta, lambda_A, Text,
66             t, lambda_D, Ntot, Pext, T, x_nodes, n,aE,aW,aP,bP,told);
67         //coefficients dels nodes interns
68         coeff_int( Ntot, Na, Nb, Nc, Nd, Nf, told, t,x, x_nodes, lambda_A, lambda_B,
69             lambda_C, lambda_D, lambda_F, H, W,T, rhoA, rhoB, rhoC, rhoD, rhoF,
70             n,ef,numfin,Text,Tsky, Pext,vext,epsilon,sigma,deltat,alpha_ext,
71             Qrad,QconvNat,beta,aE,aW,aP,bP);
72         //3.3.SOLVER
73         solve(Told, Tval, Ntot, aE, aW,aP, bP, T, n);
74         for (int i=0; i<Ntot;i++) {
75             diff = fabs(Tval[i] - Told[i]);
76             if (diff>Mdiff)
77                 Mdiff=diff;
78         }
79         if(Mdiff<maxdiff)
80             maxdiff=Mdiff;
81         update(T, Tval, n, Ntot);
82     }
83     n++;
84     for(int i=1;i<nmax;i++){
85         cFinal = fabs(T[i]-T[i-1]);
86         if(cFinal>cFi)
87             cFi = cFinal;
88     }
89     if(cFi<critFinal)
90         critFinal=cFi;
91     //4.CRITERI DE CONVERGENCIA
92     if(n>=nmax||critFinal<delta)
93         crit = 1;
94 }
95
96 //5.IMPRESSIO DE RESULTATS I CALCULS FINALS
97 //el codi d'impressio no esta inclos aqui ja que no s'ha fet en C, sino en Python
98
99 free(x); free(x_nodes); free(T); free(Tcomp); free(aE); free(aW); free(bP);
100 free(aP); free(Tval); free(Told); free(Qrad); free(QconvNat);
101
102 //6.FI
103 return 0;
104 }

```

B Previous calculations code

headers/presolver.h

```

1  #ifndef PRESOLVER_H
2  #define PRESOLVER_H
3
4  void FV(double W, double h, double Lf, double *F23,
5         double *F24, double *F2sky, double *F32,
6         double *F34, double *F3sky, double *F42,
7         double *F43, double *F4sky);
8
9  void mesh(double *x, int n_total, int Na, int Nb, int Nc,
10          int Nd, int Nf, double delta_xA, double delta_xB,
11          double delta_xC, double delta_xD, double delta_xF,
12          double *x_nodes);
13
14  double ConvecF(double Tw, double Text, double Pext, double x,
15               double vext);
16
17  double ConvecNatu(double Tw, double Text, double Pext, double x);
18
19  double q_A(double t); double q_D(double t);
20  double q_VA(double t); double q_VB(double t);
21  double q_VC(double t); double q_VD(double t);
22
23  double Cpa(double T); double Cpb(double T);
24  double Cpc(double T); double Cpd(double T);
25  double Cpf(double T);
26
27  #endif

```

sources/presolver.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <math.h>
4  #include "../headers/presolver.h"
5
6  void FV(double W, double h, double Lf, double *F23,
7         double *F24, double *F2sky, double *F32,
8         double *F34, double *F3sky, double *F42,
9         double *F43, double *F4sky){
10     double A2 = W*h, A3 = W*Lf, A4 = W*h;
11     double theta = atan(h/Lf);
12
13     *F23 = ((Lf+h)/(Lf/cos(theta)))/(2*h);
14     *F24 = ((Lf/cos(theta)*2)-(2*Lf))/(2*h);
15     *F2sky = 1.0-*F23-*F24;
16     *F32 = (A2/A3)*(*F23);
17     *F34 = (((Lf+h)-(Lf/cos(theta)))/(2*Lf));
18     *F3sky = 1.0-*F32-*F34;
19     *F42 = (A2/A4)*(*F24);
20     *F43 = (A3/A4)*(*F34);
21     *F4sky = 1.0-*F42-*F43;
22 }
23
24
25 void mesh(double *x, int n_total, int Na, int Nb, int Nc,
26          int Nd, int Nf, double delta_xA, double delta_xB,
27          double delta_xC, double delta_xD, double delta_xF,
28          double *x_nodes){
29     double dx;
30     x[0] = 0;
31     x_nodes[0] = 0;
32     for(int i=1; i<n_total+1; i++){
33         if(i<=Na)
34             dx = delta_xA;

```

```

35     else if(i<=Na+Nb)
36         dx = delta_xB;
37     else if(i<=Na+Nb+Nf)
38         dx = delta_xF;
39     else if(i<=Na+Nb+Nf+Nc)
40         dx = delta_xC;
41     else
42         dx = delta_xD;
43     x[i]=x[i-1]+dx;
44     x_nodes[i] = (x[i]+x[i-1])/2;
45 }
46 x_nodes[n_total+1]=x[n_total];
47 }
48
49 double ConvecF(double Tw,double Text,double Pext,double x,
50     double vext){
51     double mu, Pr, alpha_F, Re, Nu;
52     double Tm = (Tw+Text)/2;
53     double cp=1034.09-2.849e-1*Tm+7.817e-4*pow(Tm,2)-4.971e-7
54         *pow(Tm,3)+1.077e-10*pow(Tm,4);
55     double rho = Pext/(287*Tm);
56     double lamb = (2.648e-3*pow(Tm,0.5))/(1+((245.4/Tm))*(pow(10,-12/Tm)));
57
58     if(Tm<1100){
59         double C1 = 1.458e-6, S = 110.4;
60         mu = (C1*pow(Tm,1.5))/(Tm+S);
61         Pr = mu*cp/lamb;
62     }
63
64     else{
65         mu = (2.5393e-5*(sqrt(Tm/273.15)))/(1+(122/Tm));
66         Pr = 0.71;
67     }
68
69     Re =(rho*vext*x)/mu;
70     if(Re<5e5)
71         Nu = 0.332*pow(Re,1/2)*pow(Pr,1/3);
72     else
73         Nu = 0.029*pow(Re,4/5)*pow(Pr,1/3);
74
75     alpha_F =(Nu*lamb)/x;
76     return (alpha_F);
77 }
78
79 double ConvecNatu(double Tw,double Text,double Pext,double x){
80     double g=9.81, mu,Pr,Gr,Ra,C,n,k, alpha_nat,Nu;
81     double Tm =(Tw+Text)/2;
82     double cp=1034.09-2.849e-1*Tm+7.817e-4*pow(Tm,2)-4.971e-7
83         *pow(Tm,3)+1.077e-10*pow(Tm,4);
84     double rho = Pext/(287*Tm);
85     double lamb = (2.648e-3*pow(Tm,0.5))/(1+((245.4/Tm))*(pow(10,-12/Tm)));
86
87     if(Tm<1100){
88         double C1 =1.458e-6, S = 110.4;
89         mu =(C1*pow(Tm,1.5))/(Tm+S);
90         Pr =(mu)*cp/lamb;
91     }
92     else{
93         mu = (2.5393e-5*(sqrt(Tm/273.15)))/(1+(122/Tm));
94         Pr = 0.71;
95     }
96     Gr = (g*(1/Tm)*pow(rho,2)*abs(Tw-Text)*pow(x,3))/pow(mu,2);
97     Ra = Pr*Gr;
98     if(Ra<pow(10,9)){
99         C =0.8;
100         n =1/4;
101         k =pow((1+pow((1+1/(sqrt(Pr))),2)), -1/4);
102     }

```

```
103     else{
104         C =0.0246;
105         n =2/5;
106         k = pow((pow(Pr,1/6)/(1+0.494*pow(Pr,2/3))),2/5);
107     }
108     Nu =(C*pow(Ra,n))*k;
109     alpha_nat =(Nu*lamb)/x;
110     return(alpha_nat);
111 }
112
113 double q_A(double t){
114     return 1000*sin(M_PI*t/60);
115     //return 0;
116 }
117
118 double q_VA(double t){
119     return 1e5;
120     //return 0;
121 }
122
123 double q_VB(double t){
124     return 5e4;
125     //return 0;
126 }
127
128 double q_VC(double t){
129     return 2e4;
130     //return 0;
131 }
132
133 double q_VD(double t){
134     return 1e4;
135     //return 0;
136 }
137
138 double q_D(double ){
139     return 5e4;
140     //return 0;
141 }
142
143 //Cp -> 0.117*T+0.0004*T^2
144 double Cpa(double T){
145     return 500;
146 }
147
148 double Cpb(double T){
149     return 700;
150 }
151
152 double Cpc(double T){
153     return 250;
154 }
155
156 double Cpd(double T){
157     return 200;
158 }
159
160 double Cpf(double T){
161     return 600;
162 }
163 //cp = 417.1+1.55*T-0.001183*T^2+3.86e-7*T^3
```

C Coefficient calculation code

headers/coeff.h

```

1  #ifndef COEFF_H
2  #define COEFF_H
3
4  void clear(double *x);
5
6  void coeff_boundary(double H, double W,
7                     double beta, double lambda_A, double Text, double t, double lambda_D,
8                     int Ntot, double Pext, double **T, double *x_nodes, int n,
9                     double *aE, double *aW, double *aP, double *bP, double told);
10
11 void coeff_int(int Ntot, int Na, int Nb, int Nc, int Nd, int Nf, double told, double t,
12              double *x, double *x_nodes, double lambda_A, double lambda_B, double lambda_C,
13              double lambda_D, double lambda_F, double H, double W, double **T,
14              double rhoA, double rhoB, double rhoC, double rhoD, double rhoF, int n,
15              double ef, int numfin, double Text, double Tsky, double Pext, double vext,
16              double epsilon, double sigma, double deltat, double alpha_ext, double *Qrad,
17              double *QconvNat, double beta, double *aE, double *aW, double *aP,
18              double *bP);
19
20 #endif

```

sources/coeff.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <math.h>
4  #include "../headers/coeff.h"
5  #include "../headers/presolver.h"
6
7  void clear(double *x){
8      for(int i=0; i<sizeof(x); i++)
9          x[i]=0;
10 }
11
12 void coeff_boundary(double H, double W,
13                   double beta, double lambda_A, double Text, double t, double lambda_D,
14                   int Ntot, double Pext, double **T, double *x_nodes, int n,
15                   double *aE, double *aW, double *aP, double *bP, double told){
16     //node inicial paret
17     double dPEini = x_nodes[1] - x_nodes[0];
18     double Seini = H*W;
19     double alpha_natin = ConvecNatu(T[n+1][0], Text, Pext, H);
20     aE[0] = ((beta*lambda_A)/dPEini)*Seini;
21     aW[0] = 0;
22     aP[0] = ((beta*lambda_A)/dPEini)*Seini+beta*alpha_natin*Seini;
23     bP[0] = beta*q_A(t)*Seini+(1-beta)*(q_A(told)+lambda_A*((T[n][1]-T[n][0])/dPEini)
24           +alpha_natin*(Text-T[n][0]))*Seini+alpha_natin*beta*Text*Seini;
25
26     //node final paret
27     double dPW = x_nodes[Ntot-1]-x_nodes[Ntot-2];
28     double alpha_natout = ConvecNatu(T[n+1][Ntot-1], Text, Pext, H);
29     aE[Ntot-1] = 0;
30     aW[Ntot-1] = beta*(lambda_D/dPW)*H*W;
31     aP[Ntot-1] = beta*(lambda_D/dPW)*H*W+alpha_natout*beta*H*W;
32     bP[Ntot-1] = (1-beta)*((-lambda_D/dPW)*(T[n][Ntot-1]-T[n][Ntot-2]))+q_D(told)
33           -alpha_natout*(T[n][Ntot-1]-Text))*H*W+q_D(t)*H*W*beta
34           +alpha_natout*beta*Text*H*W;
35 }
36
37 void coeff_int(int Ntot, int Na, int Nb, int Nc, int Nd, int Nf, double told, double t,
38              double *x, double *x_nodes, double lambda_A, double lambda_B, double lambda_C,
39              double lambda_D, double lambda_F, double H, double W, double **T,
40              double rhoA, double rhoB, double rhoC, double rhoD, double rhoF, int n,
41              double ef, int numfin, double Text, double Tsky, double Pext, double vext,

```

```

42         double epsilon, double sigma, double deltat, double alpha_ext, double *Qrad,
43         double *QconvNat, double beta, double *aE, double *aW, double *aP,
44         double *bP){
45 //calcul de distancies, superficies, propietats,... corresponents al node
46 double dPE, dPW, dPe, deE, dPw, dwW, Se, Sw, Vp, qv, qv_old, Sconv, Cpp, rho, nat;
47 double lambdaE, lambdaW, Sconv2, alpha_convNat;
48 for(int i=1;i=Ntot-1;i++){
49     dPE = x_nodes[i+1]-x_nodes[i];
50     dPW = x_nodes[i]-x_nodes[i-1];
51     if(i==Na||i==Na+1||i==Na+Nb||i==Na+Nb+1||i==Na+Nb+Nf+1||i==Na+Nb+Nf||
52     i==Na+Nb+Nc+Nf+1||i==Na+Nb+Nc+Nf){
53         dPe=x[i]-x_nodes[i];
54         deE=x_nodes[i+1]-x[i];
55         dPw =x_nodes[i]-x[i-1];
56         dwW =x[i-1]-x_nodes[i-1];
57     }
58     if(i<Na){
59         lambdaE = lambda_A;
60         lambdaW = lambda_A;
61         Se = H*W;
62         Sw = H*W;
63         Vp = (x[i]-x[i-1])*W*H;
64         qv = q_VA(t);
65         qv_old = q_VA(told);
66         Sconv = 0;
67         Cpp = Cpa((T[n+1][i]+T[n][i])/2);
68         rho = rhoA;
69         nat = 1;
70     }
71     else if(i==Na){
72         lambdaE = dPE/((dPe/lambda_A)+(deE/lambda_B));
73         lambdaW = lambda_A;
74         Se = H*W;
75         Sw = H*W;
76         Vp = (x[i]-x[i-1])*W*H;
77         qv = q_VA(t);
78         qv_old = q_VA(told);
79         Sconv = 0;
80         Cpp = Cpa((T[n+1][i]+T[n][i])/2);
81         rho = rhoA;
82         nat =1;
83     }
84     else if(i==Na+1){
85         lambdaE = lambda_B;
86         lambdaW = dPW/((dPw/lambda_B)+(dwW/lambda_A));
87         Se = H*W;
88         Sw = H*W;
89         Vp = (x[i]-x[i-1])*W*H;
90         qv = q_VB(t);
91         qv_old = q_VB(told);
92         Sconv = 0;
93         Cpp = Cpb((T[n+1][i]+T[n][i])/2);
94         rho = rhoB;
95         nat = 1;
96     }
97     else if(Na+1<i<Na+Nb){
98         lambdaE = lambda_B;
99         lambdaW = lambda_B;
100        Se = H*W;
101        Sw = H*W;
102        Vp = (x[i]-x[i-1])*W*H;
103        qv = q_VB(t);
104        qv_old = q_VB(told);
105        Sconv = 0;
106        Cpp = Cpb((T[n+1][i]+T[n][i])/2);
107        rho = rhoB;
108        nat = 1;
109    }

```

```

110     else if(i==Na+Nb){
111         lambdaE = dPE/((dPe/lambda_B)+(deE/lambda_F));
112         lambdaW = lambda_B;
113         Se = ef*W*numfin;
114         Sw = H*W;
115         Sconv = H*W-numfin*ef*W;
116         Vp = (x[i]-x[i-1])*W*H;
117         qv = q_VB(t);
118         qv_old = q_VB(told);
119         Cpp = Cpb((T[n+1][i]+T[n][i])/2);
120         rho = rhoB;
121         nat = 1;
122     }
123     else if(i==Na+Nb+1){
124         lambdaE = lambda_F;
125         lambdaW = dPW/((dPw/lambda_F)+(dwW/lambda_B));
126         Se = ef * W * numfin;
127         Sw = ef * W * numfin;
128         Sconv = 2* numfin *(x[i]-x[i-1])*W;
129         Vp =(x[i] - x[i-1]) * W * ef * numfin;
130         qv = 0;
131         qv_old = 0;
132         Cpp = Cpf((T[n+1][i]+T[n][i])/2);
133         rho = rhoF;
134         nat =0;
135     }
136     else if(Na + Nb +1 <i < Na + Nb + Nf){
137         lambdaE = lambda_F;
138         lambdaW = lambda_F;
139         Se = ef * W * numfin;
140         Sw = ef * W * numfin;
141         Sconv = 2 * numfin * ( x[ i ] - x[ i - 1] ) * W;
142         Vp = ( x [ i ] - x [ i - 1] ) * W * ef * numfin;
143         qv = 0;
144         qv_old = 0;
145         Cpp = Cpf((T[ n +1][ i ]+ T[ n ][ i ] ) /2);
146         rho = rhoF;
147         nat =0;
148     }
149     else if(i == Na + Nb + Nf){
150         lambdaE = dPE /(( dPe / lambda_F ) +( deE / lambda_C ) );
151         lambdaW = lambda_F;
152         Se = ef * W * numfin;
153         Sw = ef * W * numfin;
154         Sconv = 2 * numfin * ( x[ i ] - x[ i - 1] ) * W;
155         Vp = ( x[ i ] - x[ i - 1] ) * W * ef * numfin;
156         qv = 0;
157         qv_old = 0;
158         Cpp = Cpf(( T[ n +1][ i ]+ T[ n ][ i ] ) /2);
159         rho = rhoF;
160         nat =0;
161     }
162     else if (i == Na + Nb + Nf +1){
163         lambdaE = lambda_C;
164         lambdaW = dPW /(( dPw / lambda_C ) +( dwW / lambda_F ) );
165         Se = H * W;
166         Sw = ef * W * numfin;
167         Sconv = H *W - numfin * W * ef;
168         Vp = ( x[ i ] - x[ i - 1] ) * W * H;
169         qv = q_VC ( t );
170         qv_old = q_VC ( told );
171         Cpp = Cpc (( T[ n +1][ i ]+ T[ n ][ i ] ) /2);
172         rho = rhoC;
173         nat =1;
174     }
175     else if(Na + Nb + Nf +1 <i < Na + Nb + Nf + Nc){
176         lambdaE = lambda_C;
177         lambdaW = lambda_C;

```

```

178     Se = H * W;
179     Sw = H * W;
180     Vp =( x[ i ] - x[ i - 1]) * W * H;
181     qv = q_VC ( t );
182     qv_old = q_VC ( told );
183     Sconv =0;
184     Cpp = Cpc (( T[ n +1][ i ]+ T[ n ][ i ]) /2);
185     rho = rhoC;
186     nat =1;
187 }
188 else if (i == Na + Nb + Nf + Nc){
189     lambdaE = dPE /(( dPe / lambda_C ) +( deE / lambda_D ));
190     lambdaW = lambda_C;
191     Se = H * W;
192     Sw = H * W;
193     Vp =( x[ i ] - x[ i - 1]) * W * H;
194     qv = q_VC ( t );
195     qv_old = q_VC ( told );
196     Sconv =0;
197     Cpp = Cpc (( T[ n +1][ i ]+ T[ n ][ i ]) /2);
198     rho = rhoC;
199     nat =1;
200 }
201 else if (i == Na + Nb + Nf + Nc +1){
202     lambdaE = lambda_D;
203     lambdaW = dPW /(( dPw / lambda_D ) +( dwW / lambda_C ));
204     Se = H * W;
205     Sw = H * W;
206     Vp = ( x[ i ] - x[ i - 1]) * W * H;
207     qv = q_VD ( t );
208     qv_old = q_VD ( told );
209     Sconv =0;
210     Cpp = Cpd (( T[ n +1][ i ]+ T[ n ][ i ]) /2);
211     rho = rhoD;
212     nat =1;
213 }
214 else{
215     lambdaE = lambda_D;
216     lambdaW = lambda_D;
217     Se = H * W;
218     Sw = H * W;
219     Vp = ( x[ i ] - x[ i - 1]) * W * H;
220     qv = q_VD ( t );
221     qv_old = q_VD ( told );
222     Sconv =0;
223     Cpp = Cpd (( T[ n +1][ i ]+ T[ n ][ i ]) /2);
224     rho = rhoD;
225     nat =1;
226 }
227 //radiacio i conveccio natural
228 Sconv2 =( x[ i ] - x[i -1])*W*nat;
229 alpha_convNat = ConvecF ( T[ n + 1][ i ],Text ,Pext ,x_nodes[i] , vext );
230 Qrad[ i ] = Sconv2 * epsilon * sigma * ( pow(T[ n + 1][ i ],4)-pow(Tsky,4));
231 QconvNat[ i ] = Sconv2 * alpha_convNat * ( T[ n + 1][ i ]- Text );
232
233 //calcul de coeficients dels nodes interns
234 aE[i]= beta * ( Se / dPE ) * lambdaE;
235 aW[ i ]= beta * ( Sw / dPW ) * lambdaW;
236 aP[ i ]= aE[ i ]+ aW[ i ]+( rho * Vp * Cpp ) / deltat + alpha_ext *Sconv * beta +
237     alpha_convNat * beta * Sconv2 + beta *epsilon * sigma * pow(T[ n +1][ i ],3)*
Sconv2;
238 bP[ i ]= beta * ( qv * Vp ) +((( rho * Vp * Cpp ) / deltat ) * T[ n ][ i ])+
239     (1 - beta ) * ( qv_old *Vp +(( lambdaE *(T[ n ][ i +1] - T[ n ][ i ])*Se)/
240     dPE)-(( lambdaW *( T[ n ][ i ] - T[ n ][ i -1]) * Sw ) /dPW ) -
241     ( alpha_ext *(T[ n ][ i ] - Text ) * Sconv ) -(alpha_convNat *(T[ n ][ i ] -
242     Text ) * Sconv2 ) -( epsilon *sigma *( pow(T[ n ][ i ],4) - pow(Tsky,4)) *
Sconv2 ) ) +
243     alpha_ext *Text * Sconv * beta + alpha_convNat * beta * Text * Sconv2 +

```

```
244         beta * epsilon * sigma * pow(Tsky,4)* Sconv2;  
245     }  
246 }
```

D Solver code

headers/solver.h

```
1  #ifndef SOLVER_H
2  #define SOLVER_H
3
4  void solve(double *Told, double *Tval, int Ntot, double *aE, double *aW,
5            double *aP, double *bP, double **T, int n);
6
7  void update(double **T, double *Tval, int n, int Ntot);
8
9  #endif
```

sources/solver.c

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <math.h>
4  #include "../headers/solver.h"
5
6  void solve(double *Told, double *Tval, int Ntot, double *aE, double *aW,
7            double *aP, double *bP, double **T, int n){
8      for(int i=0; i<Ntot; i++){
9          Told[i] = T[n+1][i];
10         Tval[0] = (aE[0] * Told[1] + bP[0]) / aP[0];
11         for(int i=1; i<Ntot-1; i++){
12             Tval[i] = (aE[i]*Told[i+1] + aW[i]*Tval[i-1] + bP[i]) / aP[i];
13         }
14         Tval[Ntot-1] = (aW[Ntot-1] * Tval[Ntot-2] + bP[Ntot-1]) / aP[Ntot-1];
15     }
16
17     void update(double **T, double *Tval, int n, int Ntot){
18         for(int i=0; i<Ntot; i++){
19             T[n+1][i] = Tval[i];
20         }
21     }
```